1. 地址解析：把 URL 变成 IP

- 解析 URL
  - 拆出协议（`http/https`）、域名、端口、路径、查询参数等
- DNS 查询
  - 先查浏览器缓存 → 系统缓存 → 本地 hosts → 运营商 DNS → 递归到根域名服务器等
  - 最终拿到目标服务器的 **IP 地址**

---

2. 建立连接：打通到服务器的“管道”

- TCP 三次握手
  - 客户端：SYN → 服务器：SYN+ACK → 客户端：ACK
  - 建立可靠的 TCP 连接（通常端口 80/443）
- TLS 握手（仅 https）
  - 协商加密算法、验证证书、公钥交换、生成对称密钥
  - 后续 HTTP 数据都在加密通道里传输

---

3. 发送请求：浏览器发 HTTP 请求

- 构造 HTTP 请求报文
  - 请求行：`GET /path HTTP/1.1`
  - 请求头：Host、Cookie、User-Agent、Accept、Content-Type 等
  - 请求体：一般 POST/PUT 等会有（如表单、JSON）
- 通过 TCP/TLS 管道发送给服务器

---

4. 服务器处理并返回响应

- 服务器收到请求
  - 负载均衡 → 网关/反向代理（如 Nginx） → 应用服务器（Node、Java、Go 等）
- 应用逻辑
  - 路由匹配、权限校验、读写数据库/缓存、渲染页面或返回 JSON
- 构造 HTTP 响应
  - 状态行：`HTTP/1.1 200 OK`
  - 响应头：Content-Type、Set-Cookie、Cache-Control 等
  - 响应体：HTML / JSON / 图片 / CSS / JS ...
- 通过 TCP/TLS 返回给浏览器

---

5. 浏览器开始渲染页面

5.1 解析 HTML，构建 DOM

- 字节流 → 字符串 → 词法/语法分析 → **DOM Tree**
- 遇到：
  - **外链 CSS**：发起并行请求
  - **外链 JS**：默认会阻塞解析，先下载执行（可用 `defer` / `async` 优化）

5.2 解析 CSS，构建 CSSOM

- 下载所有 CSS → 解析为 **CSSOM Tree**
- DOM + CSSOM → Render Tree（渲染树）

5.3 布局（Layout / Reflow）

- 计算每个渲染节点的 **位置和尺寸**（x、y、width、height）
- 这是“回流/重排”

5.4 绘制（Paint / Repaint）

- 按样式把文本、边框、背景、阴影等画成**位图**

5.5 分层与合成（Compositing）

- 某些节点被提升为**单独图层**（position、transform、动画等）
- GPU 对图层进行**合成**：叠加、裁剪、位移、缩放
- 画面最终输出到屏幕

---

6. 资源加载与执行

- **并行加载静态资源**：图片、CSS、JS、字体等（受同域并发连接数限制）
- JS 执行
  - 修改 DOM / 样式 → 可能触发回流/重绘/合成
- 事件绑定、首屏渲染完成
  - SPA 中还会有路由切换、首屏 hydration 等

---

7. 连接复用与后续交互

- HTTP/1.1 Keep-Alive / HTTP/2 多路复用
  - 同一 TCP 连接上复用多次请求
- 页面后续的 Ajax / Fetch / WebSocket 交互
  - 持续更新 DOM，用户看到动态内容